# Table of Contents


1. INTRODUCTION 1
    1.1 Background of the Project 1
    1.2 Problem Statement 1
    1.3 Proposed Solution 1
    2.1 Project Objectives 2
    2.2 Project Scope 2
        2.2.1 Included Scope 2
        2.2.2 Excluded Scope 3
    2.3 Stakeholder Analysis 3
        2.4 High-Level Requirements 5
            2.4.1 Functional Requirements 5
            2.4.2 Non-Functional Requirements 6
3. STATEMENT OF WORK (SOW) 7
    3.1 Project Overview 7
    3.2 Project Deliverables 7
    3.3 Project Schedule (12-Week Timeline) 8
    3.4 Roles and Responsibilities 9
    3.4.1 Resource Allocation: Team Role 9
        Hawlet Musbah: Project Manager (PM) & System Analyst 9
        Haymanot Haderu: Database Administrator (DBA) & Security Engineer 9
        Hana Workalmahu: Backend Developer 9
        Hana Mesfin: Front-End / Mobile Developer (Customer Facing) 9
        Fenet Bekele: UI/UX Designer & Station Interface Developer 10
    3.5 Resource Requirements 10
        3.5.1 Hardware Requirements 10
        3.5.2 Software Requirements 10
    3.6 Cost Estimation 11
4. RISK ASSESSMENT 12
    4.1 Introduction to Risk Management 12
    4.2 Risk Assessment Table 12
    4.3 Risk Monitoring Strategy 14
5. QUALITY ASSURANCE PLAN 15
    5.1 Quality Objectives 15
    5.2 Quality Standards 15
    5.3 Testing Strategy 16
        5.3.1 Unit Testing 16
        5.3.2 Integration Testing 17

5.3.3 System Testing 17
5.3.4 User Acceptance Testing 18
5.4 Test Cases 18
5.5 Bug Reporting and Resolution 20
6. COMMUNICATION PLAN 21
7. FEASIBILITY STUDY 22
7.1 Technical Feasibility 22
7.2 Economic Feasibility 22
7.3 Operational Feasibility 22
7.4 Schedule Feasibility 22
8. CONCLUSION 23
9. REFERENCES 23

# 1. INTRODUCTION

## 1.1 Background of the Project

Fuel distribution and queue management have become major challenges in Ethiopia due to recurring fuel shortages, wars and increasing demand for petroleum products. At fuel stations, especially in large cities such as Addis Ababa, drivers are often forced to wait in long physical queues for many hours or even days before receiving fuel. This situation affects public transportation, business activities, and the daily lives of citizens.

The current manual system creates traffic congestion, unfair distribution of fuel, and inefficient station operations. As a result, there is a need for a computerized system that can digitize reservations, manage fuel stock, and improve queue handling procedures.

## 1.2 Problem Statement

Fuel stations in Ethiopia currently lack a proper digital system to manage fuel distribution and vehicle queues effectively. Drivers spend more time waiting in physical lines without knowing when they will be served. This leads to frustration, loss of productivity, and reduced income for taxi drivers.

The problem also affects the wider community because public transportation becomes unreliable when taxis remain stuck in fuel queues. Long queues extend onto major roads, creating traffic congestion and disrupting transportation systems. In addition, the absence of a controlled system allows some vehicles to take more fuel than expected, while others fail to receive fuel at all.

Manual record management further creates inefficiency, poor accountability, and difficulty in tracking fuel stock and transactions. Therefore, a reliable digital solution is required to improve fairness, transparency, and operational efficiency in fuel distribution.

## 1.3 Proposed Solution

The proposed solution is a Fuel Queue and Distribution Management System developed using mainly relational database principles. The system introduces a digital reservation platform where drivers can register for fuel service instead of waiting in physical queues.

The system will:

- Manage digital reservations and queue positions
- Estimate waiting times automatically
- Track fuel stock availability
- Prevent multiple bookings
- Record fuel transactions
- Enforce maximum fuel limits based on vehicle type
- Automatically remove expired reservations after a grace period

The database will include entities such as stations, workers, vehicles, customers, reservations, and money transactions to ensure efficient management of station operations.

## 2.1 Project Objectives

The main objectives of the project are:

- To reduce long waiting times (physical fuel queues) at stations.
- To create a fair first-come, first-served digital reservation system for fuel distribution.
- To prevent fuel hoarding and double booking through controlled reservation policies because of the shortage.
- To digitize manual station records and transaction systems.
- To improve the fuel stock monitoring system.
- To increase operational efficiency and accountability at fuel stations.
- To provide accurate transaction logging and queue tracking for management purposes.

## 2.2 Project Scope

The system focuses on improving fuel distribution, queue management, and transaction handling for fuel stations operated by TotalEnergies.

### 2.2.1 Included Scope

The following functionalities are included within the scope of the project:

1. **Digital Reservation System**
    - Vehicle registration for fuel booking
    - Queue position assignment
    - OTP verification system
    - Waiting time estimation
    - Queue status management
2. **Fuel Stock Management**
    - Daily fuel quota tracking

    - Benzene and diesel amount monitoring
    - Automatic registration stop when stock is depleted
3. **Customer and Vehicle Management**
    - Customer registration
    - Vehicle information storage
    - Fuel limit enforcement based on vehicle type
4. **Transaction**
    - Recording fuel purchases
    - Worker verification tracking
    - Total price calculation
    - Date and time recording
5. **Station and Worker Management**
    - Managing station information
    - Worker role management
6. **Automated Queue Policies**
    - First-registered, first-served processing
    - 20-minute waiting period rule
    - Expired reservation handling

### 2.2.2 Excluded Scope

The following features are excluded from this project:

- Car maintenance and repair services
- Fuel delivery to homes or businesses
- Full online payment gateway or credit card processing
- Loyalty or reward programs
- Non-fuel retail shop management (cafés, Bonjour shops, etc.)
- Electric vehicle charging management
- Financial accounting systems beyond transaction recording

## 2.3 Stakeholder Analysis

For the 'Ethio-FuelFlow' system, the stakeholder ecosystem is multi-layered, involving government regulators, corporate leadership, and the general public. The following table provides a comprehensive analysis of these key players, their specific responsibilities, and the impact they have on the system’s design and overall success.

| Category | Stakeholder | Detailed Role & Responsibility |
|---|---|---|
| **Regulatory Body** | **Petroleum & Energy Authority (PEA)** | **Policy & Price Oversight:** They serve as the primary regulator, setting national fuel prices and the "Triage" priority levels. |
| **Corporate Client** | **TotalEnergies Marketing Ethiopia** | **Network Operations**: They will use the system's dashboard to ensure fuel reaches branches efficiently and to prevent "ghost stocks" or hoarding at the dealer level. |
| **Branch Management** | **Station Managers & Dealers** | **On-Site Supervision**: Their role involves updating the digital inventory when a tanker arrives and reconciling the system's transaction logs with physical Telebirr payment receipts. |
| **Operational Staff** | **Station Attendants (Pump Workers)** | **Direct System Users:** verify the driver's 6-digit OTP code and Digital ID before dispensing fuel, ensuring that the actual amount sold matches what is recorded in the database. |
| **Primary End-Users** | **Public & Private Drivers (Taxis, Buses, Private Cars)** | **The Service Beneficiaries:** They use the system via mobile or USSD to join digital queues, receive SMS wait-time alerts, and, for public transporters, access government-subsidised fuel prices. |
| **Technical Team** | **Development Group (Our Team)** | **System Engineers:** We are responsible for the entire Software Development Life Cycle (SDLC). |
| **Enforcement** | **Traffic & Federal Police** | **Public Safety & Integrity:** They monitor physical station locations to manage traffic flow and prevent road blockages. They will use read-only reports from the system to identify suspicious sales patterns and stop the illegal "black market" trade of fuel. |

## 2.4 High-Level Requirements

### 2.4.1 Functional Requirements

This section outlines the specific functions the system must perform, covering the entire process from initial vehicle registration to the final transaction at the pump.

Functional Requirement 1. User & Vehicle Management

- Functional Requirement -1.1: User Registration: The system shall allow Customers to create accounts and register their Vehicles, capturing Plate Numbers and Service Types (Code 1, 2, or 3).
- Functional Requirement -1.2: Profile Management: The system shall allow users to manage their profiles, including hashed password updates and contact information.
- Functional Requirement -1.3: Verification: The system shall verify that a vehicle is only registered once to prevent "double-booking" or selling queue spots.

Functional Requirement 2. Intelligent Queue & Reservation Engine

- Functional Requirement -2.1: Digital Booking: The system shall allow users to join a digital Queue for a specific fuel station based on their fuel type requirement (Benzene or Diesel).
- Functional Requirement -2.2: Tiered Prioritisation (Triage): The system shall automatically sort the queue based on the "Service_type" (Tier 1: Essential/Public Transit, Tier 2: Agriculture, Tier 3: Private).
- Functional Requirement -2.3: OTP Generation: Upon successful registration in the queue, the system shall generate a unique OTP_Code (Verification code) for the user.
- Functional Requirement -2.4: Wait Time Estimation: The system shall calculate and display the Estimated_Wait_Time for each driver based on their position in the queue.

Functional Requirement 3. Inventory & Station Management

- Functional Requirement -3.1: Live Stock Tracking: The system shall dynamically track the Benzene Stock and Diesel Stock at each Station.
- Functional Requirement -3.2: Quota Management: During reservation, the system shall subtract the requested volume from the station's total stock and stop accepting registrations once the Daily_Quota is reached.
- Functional Requirement -3.3: Emergency Reserve: The system shall maintain a dedicated portion of fuel specifically for Tier 1 (Emergency) vehicles, even if the general daily quota is finished.

Functional Requirement 4. Operational Arrival Policy (The 20-Minute Rule)

- Functional Requirement -4.1: Arrival Notification: The system shall send real-time notifications to drivers when their turn is approaching or is called.

- Functional Requirement -4.2: Grace Period Logic: The system shall initiate a 20-minute grace period if a vehicle is not at the pump when called.
- Functional Requirement -4.3: Automated Cancellation: If the 20-minute window expires without verification, the system shall automatically delete the car from the list and release the reserved fuel quota to other drivers.

Functional Requirement 5. Transaction & Worker Verification

- Functional Requirement -5.1: Worker Authentication: The system shall allow Workers to log in and associate themselves with a specific station branch.
- Functional Requirement -5.2: OTP Verification: The system shall provide an interface for Workers to enter and verify the Customer's OTP_Code before dispensing fuel.
- Functional Requirement -5.3: Sales Recording: The system shall generate a MONEY_TRANSACTION record for every successful fuel sale, linking the Worker, Reservation, and Total Price.

### 2.4.2 Non-Functional Requirements

This section outlines the essential quality benchmarks the system must meet, focusing on security, processing speed, and overall reliability to ensure a smooth and trustworthy experience for all stakeholders.

- **NFR-1 Security (Data Integrity):** To protect user privacy, the system shall store all passwords and sensitive identification data using industry-standard hashing algorithms (such as BCrypt).
- **NFR-2 Availability (Uptime):** The system shall maintain an uptime of 99.9%. Because fuel distribution is a critical national service, the platform cannot afford downtime during times of crisis.
- **NFR-3 Performance (Latency):** The system shall be highly responsive, processing OTP verifications and transaction updates in under 2 seconds to prevent traffic congestion at the fuel pumps.
- **NFR-4 Scalability:** The system must be capable of handling at least 1,000 concurrent users across various regions of Ethiopia without any degradation in performance.
- **NFR-5 Reliability (Consistency):** The system shall ensure absolute transactional consistency. The logic must guarantee that a specific fuel quota is never "double-booked" by two different vehicles.
- **NFR-6 Usability:** The user interface for Station Attendants shall be simplified and optimized for high-speed use on mobile devices and rugged tablets to accommodate a fast-paced work environment.

# 3. STATEMENT OF WORK (SOW)

## 3.1 Project Overview

The Fuel Queue and Distribution Management System is a relational database project developed for TotalEnergies Marketing Ethiopia to modernise fuel distribution during periods of fuel shortage. The system replaces inefficient physical queues with a structured digital reservation platform that improves fairness, transparency, and operational control.

The project focuses on managing fuel reservations, monitoring station fuel availability, tracking customer transactions, and enforcing fuel allocation policies based on vehicle type and service priority. Through automated queue handling, reservation validation, and stock management, the system helps reduce congestion, improve service delivery, and ensure efficient fuel utilization.

The database system supports essential sectors such as public transportation and emergency services by implementing priority-based allocation rules. Additionally, it provides accurate digital records for station workers, reservations, vehicles, customers, and financial transactions, enabling better operational oversight and decision-making.

Overall, the project demonstrates how database systems can address real-world socioeconomic and operational challenges within Ethiopia's fuel distribution sector.

## 3.2 Project Deliverables

**The following items will be submitted upon the completion of this project:**

1. **Documentation:**

- Completed Software Requirements Specification (SRS) document :
  A comprehensive document that acts as the "source of truth" for the project. It details all functional requirements, non-functional requirements, and specific use cases for every user role.
- Software Design Document (SDD): This provides the internal blueprint of the system. It includes the System Architecture and the Database Schema/ERD.
- Final Project Report: A high-level summary of the entire development journey. It documents the project's successes, the technical challenges faced by the team, and a final evaluation of whether the original objectives were met.

2. **Design Artefacts:**

- High-fidelity UI/UX wireframes and interactive prototypes: these are interactive "clickable" designs. They demonstrate the visual layout, color schemes, and navigation flow of the application before the final code is written, ensuring a user-centric design approach.

3. **Software Product:**

- Complete Source Code: The full, commented codebase for the application. This will be hosted in a version-controlled environment (GitHub/GitLab).
- Deployed version of the application: A functional, "production-ready" version of the software hosted on a web server.
- The SQL database initialisation scripts.

4. **Quality Assurance:**

- Test Case Report: A formal document proving the reliability of the software. It includes a series of "pass/fail" tests covering every major feature, ensuring that the code is free of critical bugs and that all user inputs are handled correctly.

## 3.3 Project Schedule (12-Week Timeline)

| phase | Task | Duration | Main Deliverables |
|---|---|---|---|
| Phase 1:Initiation | Problem identification, Group formation, research, and Project Proposal submission. | Week 1 | Approved Project Idea & Pre-SRS Document. |
| Phase 2: Requirement analysis | Detailed gathering of user needs, defining functional/non-functional requirements. | Week 2 - 3 | Final Software Requirements Specification (SRS). |
| Phase 3: System Design | Creating UI/UX wireframes (Figma), designing the Database Schema (ERD), and choosing the tech stack (e.g., Python, SQL). | Week 4 - 5 | Interactive UI Prototypes & Database Blueprint |
| Phase 4: Core Development | Sprint 1: Backend & Database setup.<br>Sprint 2: Frontend development.<br>Sprint 3: Integration of Frontend and Backend. | Week 6 - 10<br>Sprint 1 week 6 and 7<br>Sprint 2 week 8 and 9<br>Sprint 3 week 10 | Functional "Alpha" version of the software |
| Phase 5: Testing and QA | Unit testing for code errors, security checks, and User Acceptance Testing (UAT) to ensure features meet the SRS. | Week 11 | Test Case Report & Bug-free "Beta" version. |
| Phase 6:Development and Handover | Hosting the app, preparing the User Manual, and final presentation/demo. | Week 12 | Final Deliverable: Source Code & Project Report |

## 3.4 Roles and Responsibilities

### 3.4.1 Resource Allocation: Team Role

**Hawlet Musbah: Project Manager (PM) & System Analyst**

- **Primary Focus:** Project execution timeline, system logic, documentation, and stakeholder requirements.
- **Detailed Tasks:**
    - Drafts and updates the Statement of Work (SOW), System Requirement Specifications (SRS), and final reports.
    - Designs the logical workflow of the queue management system and notification rules (e.g., defining how many minutes before a turn a user should be notified).
    - Acts as the liaison between the team and your academic advisor, tracking progress to avoid bottleneck delays.

**Haymanot Haderu: Database Administrator (DBA) & Security Engineer**

- **Primary Focus:** Data modelling, relational schema architecture, and data security.
- **Detailed Tasks:**Designs and normalises the relational database schema using **PostgreSQL** or **MySQL** (handling entities like Users, Vehicles, Reservations, Transactions, and Fuel_Inventory). Writes and optimises complex SQL scripts, relational joins, and transaction logic to ensure that once a vehicle receives its daily quota, it cannot exploit a loop and double-book on the same day. Implements data security standards, ensuring passwords and transaction records are encrypted.

**Hana Workalmahu: Backend Developer**

- **Primary Focus:** Server-side logic, vehicle business rules, and external API integration.
- **Detailed Tasks:**
    - Builds the core application logic using frameworks like Node.js (Express), Python (Django/FastAPI), or Java (Spring Boot).
    - Implements the rules engine that calculates specific fuel caps based on vehicle parameters (e.g., Baajaj = Max 15L, Automobile = Max 40L per turn).
    - Integrates third-party service APIs, specifically local payment systems (like Chapa, Telebirr, or CBE Birr) and an SMS gateway (like Ethio Telecom's bulk SMS or a service like TeleSign/Twilio) for sending real-time turn notifications.

**Hana Mesfin: Front-End / Mobile Developer (Customer Facing)**

- **Primary Focus:** Driver booking experience, usability, and responsiveness.
- **Detailed Tasks:**

    - Develops a responsive mobile web application or native Android app using React, Flutter, or standard HTML5/CSS Grid/JS.
    - Builds intuitive interfaces for user registration, vehicle selection, calendar booking views, and a visual countdown timer showing queue progress.
    - Generates a secure QR code on the user UI that encapsulates the booking ID for pump attendants to easily scan.

**Fenet Bekele: UI/UX Designer & Station Interface Developer**

- **Primary Focus:** Attendant/Manager interfaces and visual design consistency.
- **Detailed Tasks:**
    - Designs high-fidelity wireframes and user prototypes for all screens, focusing on high-visibility layouts for pump workers operating out in daylight.
    - Develops the station Manager Dashboard (for tracking metrics) and the Attendant Verification Interface (a lightweight, low-bandwidth UI used at the pump to mark a reservation as "Served").
    - Conduct usability testing simulations within your group to find bottlenecks in the booking flow.

## 3.5 Resource Requirements

### 3.5.1 Hardware Requirements

- **Developer Laptops:** Standard personal computers (e.g., Core i5 / AMD Ryzen 5, 8GB-16GB RAM) used by the 5 students to write and compile code locally.
- **Testing Mobile Devices:** Android smartphones (to test how the responsive booking app and worker verification views look on actual screens).
- **Local Test Server:** An existing machine used as a staging environment to run the backend and database simultaneously during group testing.

### 3.5.2 Software Requirements

- **Database Management System:** PostgreSQL (Open-source, highly reliable for handling ACID transactions and complex relational queries).
- **Backend Environment:** Node.js or Python FastAPI (Free, lightweight, fast execution for asynchronous notifications).
- **Frontend Framework:** Flutter or React (Enables responsive, mobile-first design so drivers can use it seamlessly on any phone).
- **Development Tools:** Git/GitHub (Version control for 5 people collaborating), VS Code (Code editor), pgAdmin 4 (Database administration GUI).
- **APIs:** Mock or sandbox environments for Telebirr/Chapa integration and a trial bulk SMS provider network.

## 3.6 Cost Estimation

| Resource Category | Estimated Cost (ETB) | Economic Justification & Breakdown |
|---|---|---|
| Development Software & Frameworks | 0 ETB | 100% Free / Open Source: Using PostgreSQL, VS Code, Git, Node.js, and Linux removes software licensing expenses entirely. |
| Domain Registration & Cloud Hosting | 6,000 – 12,000 ETB | Local Web Hosting: Standard local VPS hosting and a .com or .et domain via providers like Yegara Host or cloud providers for a 6-to-12-month development cycle. Crucial for testing real SMS callbacks and mobile payment webhooks. |
| Bulk SMS Gateway API Credits | 2,500 – 5,000 ETB | Notification Testing: Sending notifications to test queues requires buying local bulk SMS credit bundles. At current rates, this allows for several thousand automated transactional test SMS messages. |
| Payment Gateway Sandbox Integration | 0 ETB | Developer Sandboxes: Services like Chapa or Telebirr integrations offer free merchant testing accounts for development environments. Commercial commission fees apply only when going live. |
| Mobile Data & Internet Connectivity | 1,000 – 15,000 ETB | Team Collaboration Data: Internet costs in Addis Ababa for 5 students over a few months of intense development. This covers heavy downloads of packages (NPM, Docker images, libraries) and remote team syncs. |
| Backup Hardware Storage | 3,500 – 7,500 ETB | Data Redundancy: A shared external 1TB HDD or reliable high-speed flash storage to maintain unified codebase backups, assets, and database dump snapshots outside of cloud repositories. |
| Total Estimated Project Budget | 22,000 – 39,500 ETB | Highly Rational for Students: By relying on personal laptops and free open-source utilities, the entire budget is confined to operational connectivity, hosting infrastructure, and communications testing. |

# 4. RISK ASSESSMENT

## 4.1 Introduction to Risk Management

Risk management is the systematic process of identifying, analysing, evaluating, and addressing potential threats that could negatively affect a project's objectives. For a project like the Ethio-FuelFlow system, proactive risk management is especially important because failure can directly disrupt essential fuel services for the public, emergency responders, and commercial transportation across Ethiopia.

The sections below categorise all identified risks for the Ethio-FuelFlow system, assess their probability and potential impact, and define concrete mitigation strategies.

## 4.2 Risk Assessment Table

The table below summarises the identified project risks and mitigation strategies.

| Risk ID | Risk Description | Category | Likelihood (1–5) | Impact (1–5) | Risk Level | Mitigation Strategy |
|---|---|---|---|---|---|---|
| **R-01** | Station owners / TotalEnergies dealers may refuse to adopt the digital system, blocking rollout at key sites. | Project – Stakeholder | 4 | 5 | **Critical** | Conduct early engagement workshops with TotalEnergies management. Involve station dealers in pilot testing and demonstrate efficiency gains before full rollout. |
| **R-02** | The 12-week development schedule may prove insufficient, causing phase delays or incomplete features. | Project – Schedule | 3 | 4 | **High** | Strictly follow the sprint-based timeline in Section 3.3. Flag blockers to the instructor immediately. Prioritise core features; defer non-critical features if needed. |

| | | | | | | |
|---|---|---|---|---|---|---|
| **R-03** | Unreliable or absent internet at many stations prevents real-time OTP delivery, queue updates, and stock tracking. | Technical – Infrastructure | 5 | 5 | **Critical** | Design a USSD fallback for feature phones. Implement offline queue caching that syncs when connectivity is restored. Test under simulated outage conditions. |
| **R-04** | Simultaneous reservations from 143+ stations during peak shortage may overload the server, causing crashes. | Technical – Scalability | 4 | 5 | **Critical** | Architect for 10,000+ concurrent users (NFR-4). Apply database connection pooling and system reliability . Conduct load tests before deployment. |
| **R-05** | The wait-time algorithm may produce inaccurate results due to no-shows, emergency insertions, or variable fill times. | Technical – Algorithm | 4 | 3 | **High** | Implement dynamic recalculation after each completed or cancelled reservation. Use average dispensing time per vehicle type as a calibrated baseline. |
| **R-06** | Low smartphone penetration among taxi and bus drivers makes the mobile app inaccessible to core users. | Business – Market | 5 | 4 | **Critical** | Develop a USSD-based registration option accessible on any basic phone. Partner with taxi associations (Code 3) for group enrollment sessions. |
| **R-7** | Low digital literacy among drivers and station workers leads to errors, slow processing, and poor adoption. | Business – Market | 4 | 4 | **High** | Design a simplified, icon-based attendant UI (NFR-6). Deliver short in-person training before launch. Provide a laminated quick-reference card for pump workers. |
| **R-8** | Dishonest workers may bypass OTP verification or share codes, undermining fairness and enabling black-market fuel trade. | Business – Strategic | 4 | 5 | **Critical** | Log every OTP attempt with worker ID, timestamp, and vehicle match. Flag anomalies (one worker processing unusually many transactions) for manager review. Audit trails deter corruption. |

| | | | | | | |
|---|---|---|---|---|---|---|
| **R-9** | TotalEnergies management may withdraw institutional support mid-project due to strategic or personnel changes. | Business – Management | 2 | 5 | **High** | Formalise project sponsorship. Provide regular progress reports. Build rapport with at least two sponsor contacts to reduce single-point dependency. |
| **R-10** | Procuring internet-capable handheld devices for all station attendants may exceed the project's available budget. | Business – Budget | 3 | 4 | **High** | Assess whether existing station smartphones can be repurposed. Design the attendant UI for lightweight mobile browsers to minimise hardware requirements. |

## 4.3 Risk Monitoring Strategy

Risk management is not a one-time activity. Throughout the 12-week development lifecycle, the team will monitor and respond to risks using the following strategies:

- **Weekly Risk Reviews:** At the start of each sprint, the team reviews the Risk Assessment Table, updates likelihood and impact scores based on new information, and logs any newly identified risks.
- **Escalation Protocol:** Any risk whose level rises to Critical during development must be immediately reported to the instructor and the relevant stakeholder (e.g., TotalEnergies contact for stakeholder risks, server provider for infrastructure risks).
- **Risk Owners:** Each identified risk is assigned to a specific team member responsible for monitoring it and implementing the mitigation strategy. Ownership assignments are maintained in the team's project management log.
- **Trigger Indicators:** Observable signals are pre-defined to indicate when a risk is materialising — for example, a missed sprint milestone (R-02), a failed load test (R-05), a reported OTP bypass during UAT (R-8), or a connectivity failure during integration testing (R-03).
- **Post-Sprint Retrospectives:** At the close of each sprint, the team assesses whether any risks materialised and evaluates the effectiveness of the mitigation strategies applied, adjusting the plan accordingly.

# 5. QUALITY ASSURANCE PLAN

This section defines the quality standards, testing strategy, test cases, and bug resolution procedures that govern the Ethio-FuelFlow system throughout its development lifecycle. The goal is to ensure that every delivered feature meets the functional and non-functional requirements specified in Section 2.4 before the system is deployed

## 5.1 Quality Objectives

- **Correctness:** All core features — OTP generation and verification, tiered queue ordering, fuel stock subtraction, grace-period cancellation, and transaction recording must behave exactly as specified in the Functional Requirements .
- **Reliability:** The system must maintain reliable availability (NFR-2) and remain stable under peak load. No single-point failure should cause the entire platform to crash.
- **Performance:** OTP verifications and transaction updates must complete in under 2 seconds (NFR-3). The system must support a minimum of 1,000 concurrent users without performance degradation (NFR-4).
- **Data Integrity & Fairness:** No two vehicles may receive the same queue position (NFR-5). Fuel quota must never be double-allocated. Emergency vehicles must always be correctly prioritised above standard users.
- **Security:** All passwords and sensitive identifiers must be stored using industry-standard hashing (BCrypt — NFR-1). No plain-text credentials may appear in the database, logs, or API responses.
- **Usability:** A station attendant with basic smartphone experience must be able to complete an OTP verification without extended training (NFR-6). The USSD flow must be complete in under five menu steps.
- **Accessibility:** The system must be usable on basic feature phones via USSD in addition to the primary mobile interface, ensuring drivers without smartphones are not excluded.

## 5.2 Quality Standards

The development team will adhere to several industry-recognised standards throughout the project. ISO/IEC 25010 will guide the system's non-functional requirements covering reliability, usability, and security, while ACID compliance will be enforced on all database operations to prevent data corruption. Security practices follow the OWASP Top 10, and all passwords are

stored using cryptographic hashing via BCrypt. Code quality is maintained through GitHub peer review, requiring approval before any feature branch is merged.

## 5.3 Testing Strategy

Testing is performed across four levels during development.

| Test Level | Sprint / Phase | Scope | Tools & Method |
|---|---|---|---|
| **5.3.1 Unit Testing** | Sprints 1–2 | Individual functions tested in isolation: OTP code generation, wait-time calculation formula, queue position assignment logic, stock subtraction trigger, grace-period countdown, and password hashing. | Python unit test. Each function receives valid inputs, boundary values, and invalid/null inputs. Pass/fail results logged per function. |
| **5.3.2 Integration Testing** | Sprint 3 | End-to-end module interaction: Customer registration → vehicle linking → queue booking → OTP delivery → worker verification → MONEY_TRANSACTION creation → stock update. | API-level test scripts simulating multi-step user journeys. Database state verified after each combined operation to confirm consistency. |
| **5.3.3 System Testing** | Week 11 | Full system under realistic conditions: 20-minute auto-cancellation, duplicate booking rejection, emergency vehicle re-ordering, daily quota cut-off, USSD fallback, and simultaneous multi-station load. | Simulated load tests (1,000 concurrent users – NFR-4). OTP response time measured against the 2-second threshold (NFR-3). All NFRs validated in this phase. |
| **5.3.4 User Acceptance Testing (UAT)** | Week 11 | Real-world scenarios executed by representative end-users: a taxi driver registering via USSD, a station attendant verifying an OTP at the pump, a manager reviewing the transaction log and daily stock report. | Structured UAT sessions with taxi association representatives and station staff. Feedback captured on a standardized form. Critical usability issues are fixed before the Week 12 deployment. |

### 5.3.1 Unit Testing

Unit tests target individual functions and database procedures in isolation. The team will write test cases for every function before or immediately after coding it, following a test-driven approach where practical. Key areas of focus include:

- OTP code generation — verifying uniqueness, correct length (6 digits), and single-use enforcement.
- Wait-time calculation — testing with varying queue lengths, vehicle types, and fill-time averages.
- Queue position assignment — confirming sequential, non-duplicate assignment under normal conditions.
- Stock subtraction — verifying the correct volume is deducted upon each confirmed booking.
- Password hashing — confirming no plain-text values are stored and that hash verification works on login.
- Grace-period timer — confirming that the 20-minute countdown triggers the correct automated cancellation.

### 5.3.2 Integration Testing

Integration tests verify that separate modules work correctly when combined. The primary integration scenario for Ethio-FuelFlow is the full reservation-to-transaction pipeline:

1. Customer registers an account and links a vehicle.
2. Customer joins the digital queue for a specific station and fuel type.
3. System assigns a queue position, deducts from stock, and generates an OTP.
4. Worker logs in and is associated with the correct station branch.
5. Worker verifies the customer's OTP at the pump.
6. System creates a MONEY_TRANSACTION record and marks the reservation as complete.

After each step, the database state is inspected to confirm that records are consistent and no orphaned or contradictory entries exist.

#### 5.3.3 System Testing

System testing evaluates the complete application against all Functional and Non-Functional Requirements under conditions that simulate real deployment. Key scenarios include:

- Automated 20-minute grace-period cancellation and fuel quota release.
- Emergency vehicle insertion and correct re-ordering of the queue.

- Daily quota depletion causing the system to stop accepting new reservations.
- Simultaneous reservations from multiple stations under simulated peak load.
- USSD flow end-to-end on a simulated feature phone environment.
- Security penetration tests covering SQL injection and authentication bypass attempts.
- Performance benchmarking: OTP response time < 2 s at 1,000 concurrent users.

### 5.3.4 User Acceptance Testing

UAT is the final validation gate before deployment. Representative end-users — including taxi drivers, a station attendant, and a branch manager — execute pre-defined real-world scenarios and provide structured feedback.

UAT Scenarios:

- A taxi driver registers via USSD, joins the queue, receives an OTP via SMS, and presents it at the pump.
- A station attendant logs in, verifies a driver's OTP, and confirms that the transaction record is correctly created.
- A branch manager views the daily transaction log and fuel stock report and confirms the data is accurate and readable.
- An emergency vehicle joins a queue with Tier 3 cars and the tester confirms it moves to the front.

Feedback is collected on a standardized UAT form. All issues rated Critical or High must be resolved before the Week 12 handover. UAT sign-off from at least one stakeholder representative is required to close this phase.

## 5.4 Test Cases

The following 20 test cases cover all functional requirement areas. Each case will be executed during Week 11 testing. Results are recorded in the formal Test Case Report deliverable.

| TC ID | Module | Test Scenario | Input / Action | Expected Output | Test Level | Expected Result |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| **TC-01** | **User & Vehicle Management** | Successful customer registration | Driver submits valid name, phone, and unique plate number. | Account created; vehicle linked to customer profile; confirmation SMS sent. | Unit | **Pass** |
| **TC-02** | **Queue & Reservation** | Queue position assigned correctly | Driver joins queue at a station with 4 vehicles already registered. | Driver receives queue position 5; no other vehicle holds position 5. | Unit | **Pass** |
| **TC-03** | **Queue & Reservation** | Race condition — simultaneous registration | Two drivers submit queue requests for the same station at exactly the same time. | Each receives a unique, sequential position; no duplicate slots created. | Integration | **Pass** |
| **TC-04** | **Queue & Reservation** | OTP generation on booking | Driver successfully joins the queue. | A unique 6-digit OTP is generated and sent to the registered phone number. | Unit | **Pass** |
| **TC-05** | **Queue & Reservation** | Emergency vehicle priority | Tier 1 emergency vehicle joins a queue containing Tier 3 private cars. | Emergency vehicle is placed ahead of all non-priority entries. | Integration | **Pass** |
| **TC-06** | **Queue & Reservation** | Wait-time estimation accuracy | Driver is at position 5; average fill time per vehicle = 3 minutes. | System displays estimated wait of approximately 12 minutes. | Unit | **Pass** |
| **TC-07** | **Queue & Reservation** | Duplicate active booking blocked | Same plate number attempts a second reservation while one is already active. | System rejects with error: 'Active reservation already exists for this vehicle.' | Unit | **Pass** |
| **TC-8** | **Arrival Policy** | 20-minute grace period — cancellation | Driver does not arrive within 20 minutes of being called. | Reservation is automatically deleted; reserved fuel quota is released back to available stock. | System | **Pass** |

| | | | | | | |
|---|---|---|---|---|---|---|
| **TC-9** | **Transaction & Worker** | Worker OTP verification — correct code | Worker enters the correct 6-digit OTP for the vehicle at the pump. | Fuel dispensing is authorized; a MONEY_TRANSACTION record is created with worker ID, vehicle, volume, price, and timestamp. | Integration | **Pass** |
| **TC-10** | **Transaction & Worker** | Worker OTP verification — wrong code | Worker enters an incorrect OTP. | System rejects entry; no transaction created; worker is prompted to retry. | Unit | **Pass** |
| **TC-11** | **Security** | SQL injection attempt on login | Attacker inputs malicious SQL string in the login field. | System sanitizes input; no unauthorized access granted; error logged. | System | **Pass** |
| **TC-12** | **Performance** | OTP verification response time | Worker submits OTP verification under peak simulated load (1,000 users). | System responds and updates the transaction in under 2 seconds (NFR-3). | System | **Pass** |

## 5.5 Bug Reporting and Resolution

All defects discovered during any testing phase are logged immediately in the team's GitHub Issues tracker. Each bug report must contain:

- Test case ID and the phase in which the bug was found.
- Step-by-step reproduction instructions.
- Actual result vs. expected result.
- Severity classification (Critical, High, Medium, or Low — see table below).
- Screenshot or log extract where applicable.

Severity levels, definitions, and resolution service-level agreements (SLAs) are defined in the table below:

| Severity | Definition | Examples in Ethio-FuelFlow | Resolution SLA |
|---|---|---|---|
| **Critical** | System crash, data loss, security breach, or core feature completely broken. | Double-booking race condition; OTP bypassed; stock goes negative. | Must be fixed within 24 hours. Blocks Beta release. Escalated immediately to full team. |
| **High** | Major feature does not work correctly but the system is still partially usable. | Grace-period timer does not trigger cancellation; wrong queue position displayed. | Fixed within 48 hours. Must be resolved before Beta release. |
| **Medium** | Feature works but produces incorrect or inconsistent output in certain conditions. | Wait time estimate is off by more than 5 minutes; SMS notification delayed. | Fixed within the current sprint. Must be resolved before final handover (Week 12). |
| **Low** | Minor UI or cosmetic issue that does not affect functionality. | Label misaligned on attendant screen; typo in a confirmation message. | Logged and addressed before final handover if time permits. Does not block release. |

All bug fixes must be peer-reviewed by at least one other team member before being merged into the main branch. A bug is considered resolved only when the original test case is re-executed and passes. The final Test Case Report submitted in Week 12 must show zero open Critical or High severity issues.

# 6. COMMUNICATION PLAN

Internal Team Communication:
Daily Sync-ups: Short messages via Telegram to discuss daily tasks and progress.
Weekly Meetings: Every Saturday (in-person or via Google Meet) to review the week's milestones and update the project schedule.
Communication with Instructor (Mr. Yared):
Progress Reports: Submitted bi-weekly or as requested to show project status.
Email: Used for formal submissions and official questions.
Stakeholder Communication:
Interviews: Meeting with station managers and drivers during the Requirement Analysis phase.
Feedback Sessions: Showing prototypes to users during the Testing phase to get their input.
Documentation & Code:

GitHub: All code updates and technical issues will be tracked here.
Google Drive: A shared folder will be used for all project documents (SRS, SOW, etc.).

# 7. FEASIBILITY STUDY

## 7.1 Technical Feasibility

Skills: Our team has the necessary coding and database skills to build the system.
Tools: We are using reliable, free tools like PostgreSQL and Node.js that are perfect for this type of project.
Solutions: The system will work even during internet outages by using USSD (text-based menus) and offline saving features.

## 7.2 Economic Feasibility

Low Cost: Most of our tools are free (Open Source), so we don't need to buy expensive software licenses.
Affordable Budget: The estimated budget (22,000 – 39,500 ETB) is realistic for a student project and covers mainly hosting and internet.
Value: It saves money for taxi and bus drivers by reducing the hours they spend waiting in line instead of working.

## 7.3 Operational Feasibility

Easy to Use: We designed simple screens for station workers and a USSD option for drivers who don't have smartphones.
Better Organisation: It replaces messy, manual station paperwork with an organised digital list.

## 7.4 Schedule Feasibility

Clear Plan: We have a 12-week plan that breaks the work into small, manageable steps.
Divided Labor: Each team member has a specific role, so we can work on different parts of the system at the same time.
Realistic Deadline: We have included enough time for testing and fixing bugs before the final submission.

# 8. CONCLUSION

The Ethio-FuelFlow system provides a practical digital solution for improving fuel queue management in Ethiopia. By replacing physical queues with a digital reservation and verification system, the project improves fairness, reduces congestion, and supports more efficient fuel distribution. Based on the feasibility analysis, the project is technically, economically, operationally, and schedule feasible for development as a student software engineering project.

# 9. REFERENCES


Addis Fortune. (2024). *Navigating the gridlock: The push for digital fuel management in Ethiopia.* Addis Fortune Publishing.

Ghazal, M., Hamouda, R., & Ali, S. (2016). A smart mobile system for the real-time tracking and management of service queues. *International Journal of Computing and Digital Systems, 5*(4), 305–313.

IBM. (2023). *What is ACID compliance?* IBM Documentation. https://www.ibm.com/topics/ACID-transactions

IEEE. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE. https://ieeexplore.ieee.org/document/720574

ISO/IEC. (2011). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2011). International Organization for Standardization.

Klimek, R. (2020). Sensor-enabled context-aware and pro-active queue management systems in intelligent environments. *Sensors, 20*(20), Article 5837. https://doi.org/10.3390/s20205837

OWASP Foundation. (2021). *OWASP Top 10:2021 — The ten most critical web application security risks.* https://owasp.org/www-project-top-ten/

PostgreSQL Global Development Group. (2024). *PostgreSQL 16 documentation.* https://www.postgresql.org/docs/

TotalEnergies Marketing Ethiopia. (n.d.). *Our history and presence in Ethiopia since 1950.* Retrieved May 22, 2026, from https://totalenergies.com.et/